

MatMul+PReLU 融合算子优化方法研究

胡弘¹, 郭红阳¹, 樊翌²

(1. 北京交通大学 网络空间安全学院, 北京 100044;

2. 北京交通大学 计算机学院, 北京 100044)

摘要: 本文以华为昇腾 910B 处理器为实验平台, 基于 Ascend C 编程框架实现了 MatMul+PReLU 融合算子, 并从向量指令优化和 Tiling 切分策略两个方面进行了性能调优。实验在 $1024 \times 640 \times 256$ 规模的矩阵乘法任务上进行。结果表明, 将 4 条 PReLU 向量指令替换为单条 LeakyRelu 指令, kernel 执行时间减少了 9.03%; 调整 Tiling 切分参数后, 执行时间进一步减少了 32.91%; 将两种优化手段结合后, 最终版本相比基准加速了 1.80 倍, 有效算力从 0.81 TFLOPS 提升至 1.45 TFLOPS。

关键词: 算子融合; Ascend C; 矩阵乘法; PReLU; Tiling 优化; 昇腾处理器; 深度学习

一、引言

深度神经网络在计算机视觉、自然语言处理和语音识别等领域取得了显著的成果。随着模型规模不断增大, 计算量也在急剧增长。在典型的神经网络中, 矩阵乘法 (MatMul) 和激活函数是两个最常见的计算操作。矩阵乘法用于实现全连接层和卷积层的核心运算, 激活函数则为网络引入非线性能力。PReLU (Parametric Rectified Linear Unit) 由 He 等人^[1] 提出, 是一种广泛使用的激活函数。在实际推理或训练过程中, 矩阵乘法和激活函数通常按顺序执行: 先完成矩阵乘法, 将结果写回全局内存 (Global Memory, 简称 GM), 再从 GM 读取数据进行激活函数计算。这种方式会产生额外的内存访问开销。

算子融合是一种常用的性能优化手段。它的基本思路是将多个连续的计算操作合并到一个 kernel 中执行, 从而减少中间数据在 GM 上的读写次数^[2]。对于 MatMul+ 激活函数的场景, 融合后矩阵乘法的输出可以直接留在芯片的片上内存中, 激活函数直接在片上完成计算, 最后再将最终结果写回 GM。这样做可以将两次 GM 访问减少为一次, 降低了内存带宽的压力。TVM^[3] 等深度学习编译器已经将算子融合作为核心优化策略。Vasilache 等人^[4] 提出的 Tensor Comprehensions 框架也实现了自动化的算子融合。

华为昇腾 (Ascend) 系列 AI 处理器采用了达芬奇架构, 其 AI Core 包含 Cube 计算单元和 Vector 计算单元。Cube 单元负责矩阵乘法等高并行度的计算, Vector 单元负责激活函数等逐元素的向量运算。Ascend C 是华为为昇腾处理器提供的算子编程语言^[5], 开发者可以使用 C/C++ 风格的语法来编写自定义算子。Ascend C 提供了 Matmul 高阶 API, 封装了矩阵乘法的切分、搬运和计算逻辑, 简化了开发过程。

然而, 即使使用了高阶 API, 算子的实际性能仍然受到多个因素的影响。向量计算部分的指令选择、Tiling 切分参数的设定、数据遍历顺序等都会对最终的执行效率产生明显的影响。这些参数的选择需要结合硬件架构特点进行针对性的调优^{[6][7]}。

本文以 MatMul+PReLU 融合算子为研究对象，在昇腾 910B 处理器上进行了系统的优化实验。PReLU 的数学定义为：

$$\text{PReLU}(x) = \begin{cases} x, & x \geq 0 \\ \alpha \cdot x, & x < 0 \end{cases} \quad (1)$$

其中 α 是一个可学习的参数。当 α 为固定常数时，PReLU 退化为 LeakyReLU^[1]。

本文的主要工作包括：(1) 基于 Ascend C 编程框架，实现了 MatMul+PReLU 融合算子的基准版本；(2) 从向量指令优化和 Tiling 切分策略两个维度，设计了逐步递进的优化方案；(3) 在昇腾 910B 处理器上进行了实验验证，给出了各优化方案的性能数据和分析。

二、 相关背景

(一) 昇腾 AI 处理器架构

昇腾 910B 处理器采用达芬奇架构，每个 AI Core 包含以下关键计算单元：(1) Cube 单元，负责矩阵乘法运算，支持半精度 (float16) 的高吞吐矩阵乘加操作；(2) Vector 单元，负责向量运算，支持逐元素的加减乘除、激活函数等操作；(3) Scalar 单元，负责标量运算和流程控制。

在内存层级方面，昇腾处理器采用了多级缓存结构。从外到内依次为：Global Memory (GM，片外内存，容量大但访问延迟高)、L2 Cache、L1 Buffer、L0A/L0B/L0C Buffer 和 Unified Buffer (UB，也称为 Local Memory)。其中，Cube 单元从 L1 和 L0 读取数据进行矩阵乘法，计算结果存放在 L0C 中，再搬运到 UB 供 Vector 单元使用。Vector 单元在 UB 上完成向量计算后，结果通过 DMA 搬回 GM。

(二) Ascend C 编程模型

Ascend C 是华为推出的算子编程语言^[5]，采用了类 C++ 的语法。其核心编程概念包括以下几个方面。

1. TPipe 和 TQue 机制

Ascend C 使用 TPipe 对象管理片上内存的分配，通过 TQue (队列) 实现不同计算阶段之间的数据同步。TQue 支持 AllocTensor (申请缓冲区)、EnQue (数据入队)、DeQue (数据出队) 和 FreeTensor (释放缓冲区) 四个操作。数据在不同阶段之间通过 EnQue/DeQue 传递，框架会自动处理同步依赖关系。

2. TBuf 机制

对于不需要跨阶段传递的临时数据，Ascend C 提供了 TBuf (静态缓冲区)。通过 Get() 方法获取一块 LocalTensor 供当前阶段内部使用，不需要手动释放。TBuf 适合用作计算过程中的临时空间。

3. GlobalTensor 和 LocalTensor

GlobalTensor 表示 GM 上的数据, LocalTensor 表示片上内存中的数据。所有的计算指令只能操作 LocalTensor。数据从 GM 到片上需要通过 DataCopy 指令搬运。

4. Matmul 高阶 API

Ascend C 提供了封装好的 Matmul API, 开发者只需要设置输入矩阵、调用 Iterate() 方法即可完成矩阵乘法的分块计算。Matmul API 内部自动处理了数据的搬运、L1/L0 的 buffer 管理和 Cube 单元的调度^[6]。

(三) Tiling 切分策略

Tiling (数据分块) 是 AI 芯片上算子优化的核心技术。由于片上内存容量有限, 无法一次性容纳整个矩阵, 需要将矩阵切分为多个小块, 逐块进行计算^[7]。Tiling 策略包括两个层面。

(1) 多核切分: 将输入矩阵沿 M 和 N 方向分配到多个 AI Core 上并行计算。每个核分到的子矩阵大小为 singleCoreM $\times$ singleCoreN。

(2) 核内切分: 在每个核内部, 将子矩阵进一步切分为 baseM $\times$ baseN 的小块。每次 Iterate 调用完成一个小块的矩阵乘法。baseM 和 baseN 的选择需要考虑片上内存 (UB) 的容量限制, baseM $\times$ baseN 不能超过 UB 的可用空间。

Tiling 参数还包括 stepM 和 stepN (每次迭代在 M 和 N 方向的步进量) 以及遍历顺序 (FIRSTM 表示 M 方向优先遍历, FIRSTN 表示 N 方向优先)。这些参数的不同组合会影响数据的局部性和计算效率。

(四) 算子融合

算子融合将多个独立的计算操作合并到一个 kernel 中执行^[2]。对于 MatMul+ 激活函数的场景, 非融合方案的数据流为:

$$GM \xrightarrow{\text{搬入}} \text{Cube 计算} \xrightarrow{\text{写回}} GM \xrightarrow{\text{再搬入}} \text{Vector 计算} \xrightarrow{\text{写回}} GM$$

融合方案的数据流为:

$$GM \xrightarrow{\text{搬入}} \text{Cube 计算} \xrightarrow{\text{片上传递}} \text{Vector 计算} \xrightarrow{\text{写回}} GM$$

融合方案省去了矩阵乘法结果写回 GM 再读回的过程, 减少了一次 GM 写和一次 GM 读操作。在本文的实验中, 矩阵 C 的大小为 $1024 \times 640 \times 4 = 2,621,440$ 字节 (float32), 融合可以省去约 5MB 的 GM 访问量。

三、算子设计与实现

(一) 算子规格

本文实现的 MatMul+PReLU 融合算子的数学表达式为:

$$C = \text{PReLU}(A \times B + \text{bias}, \alpha) \quad (2)$$

其中 A 为 $M \times K$ 的输入矩阵, B 为 $K \times N$ 的权重矩阵, bias 为 $1 \times N$ 的偏置向量, C 为 $M \times N$ 的输出矩阵。实验中使用的具体参数见表 1。

表 1: MatMul+PReLU 算子输入输出规格

名称	形状	数据类型	数据格式	说明
A (输入)	[1024, 256]	float16	ND	左矩阵
B (输入)	[256, 640]	float16	ND	右矩阵
bias (输入)	[1, 640]	float32	ND	偏置
C (输出)	[1024, 640]	float32	ND	结果矩阵

PReLU 的参数 α 设为 0.5, 在算子实现中作为固定常数使用。

(二) 数据流设计

融合算子的计算流程分为三个阶段 (Stage), 如图 1 所示。

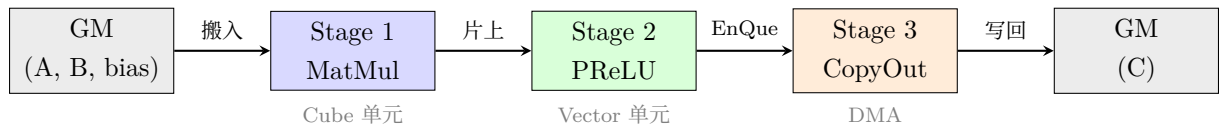


图 1: MatMul+PReLU 融合算子数据流

Stage 1 (MatMul 计算): Matmul 高阶 API 从 GM 搬入矩阵 A 、 B 和 bias 的分块数据, 在 Cube 单元上完成矩阵乘法和偏置加法。计算结果以 $\text{baseM} \times \text{baseN}$ 大小的 LocalTensor 形式输出到 UB 中。

Stage 2 (PReLU 计算): 在 Vector 单元上对 MatMul 的输出进行 PReLU 激活运算。计算完成后, 结果通过 TQue 的 EnQue 操作传递给下一阶段。

Stage 3 (CopyOut): 通过 DeQue 获取 PReLU 的计算结果, 使用 DataCopy 指令将结果搬运回 GM 的对应位置。搬运完成后通过 FreeTensor 释放缓冲区。

三个阶段在一个 while 循环中反复执行, 每次循环处理一个 $\text{baseM} \times \text{baseN}$ 的数据块, 直到当前核的全部数据处理完成。

(三) 基准实现

基准版本的核心代码框架如下:

```

1 void Process(TPipe *pipe) {
2     uint32_t computeRound = 0;
3     REGIST_MATMUL_OBJ(pipe, GetSysWorkSpacePtr(), matmulObj);
4     matmulObj.Init(&tiling);
5     matmulObj.SetTensorA(aGlobal);

```

```

6   matmulObj.SetTensorB(bGlobal);
7   matmulObj.SetBias(biasGlobal);
8   while (matmulObj.template Iterate<true>()) {
9       MatmulCompute();    // Stage 1
10      PreluCompute();     // Stage 2
11      CopyOut(computeRound); // Stage 3
12      computeRound++;
13  }
14  matmulObj.End();
15 }

```

Listing 1: 基准版本的 Process 函数

基准版本的 PReLU 计算使用了 4 条向量指令来实现：

```

1 void PreluCompute() {
2     tmpLocal = tmpBuf.Get<cType>();
3     // PReLU(x) = max(0,x) + alpha * min(0,x)
4     Mins(tmpLocal, reluOutLocal, 0, count);
5     Muls(tmpLocal, tmpLocal, alpha, count);
6     Maxs(reluOutLocal, reluOutLocal, 0, count);
7     Add(reluOutLocal, reluOutLocal, tmpLocal, count);
8     reluOutQueue.Enqueue(reluOutLocal);
9 }

```

Listing 2: 基准版本的 PReLU 实现（4 条指令）

这种实现方式需要 4 次向量指令发射，并且需要一块额外的 tmpBuf 空间来存放中间结果。基准版本的 Tiling 参数设置为：baseM=64，baseN=64，stepM=4，stepN=5，使用 2 个 AI Core，未设置遍历顺序。

四、 优化策略

本节介绍两种优化手段：向量指令优化和 Tiling 切分策略优化，以及将两者结合后的效果。

（一） 向量指令优化

1. 问题分析

基准版本使用 4 条向量指令实现 PReLU：Mins、Muls、Maxs、Add。这 4 条指令之间存在数据依赖关系（后一条指令的输入依赖前一条的输出），必须串行执行。此外，Mins 和 Muls 操作需要一块与输出矩阵等大的临时缓冲区 (tmpBuf)，占用了额外的 UB 空间。

2. 优化方案

当 PReLU 的参数 α 为固定常数时, PReLU 在数学上等价于 LeakyReLU^[1]。Ascend C 提供了 LeakyRelu 内置接口^[5], 该接口将 PReLU 的计算逻辑封装为单条指令:

```
1 void PreluCompute() {  
2     LeakyRelu(reluOutLocal, reluOutLocal, alpha, count);  
3     reluOutQueue.Enqueue(reluOutLocal);  
4 }
```

Listing 3: 优化后的 PReLU 实现 (单条指令)

这种优化带来两个好处: (1) 减少了 3 次向量指令的发射和执行开销; (2) 不再需要 tmpBuf 临时空间, 释放了 UB 资源, 为增大 Tiling 块提供了条件。

(二) Tiling 切分策略优化

1. 问题分析

基准版本的 Tiling 参数为 baseM=64、baseN=64, 单个数据块大小为 $64 \times 64 = 4096$ 个元素。对于 singleCoreM=512、singleCoreN=640 的单核工作量, 需要 $\frac{512}{64} \times \frac{640}{64} = 80$ 次迭代。每次迭代都伴随着 Tensor 的分配/释放、队列的入队/出队以及 DMA 搬运指令的发起, 这些操作构成了固定的调度开销。迭代次数越多, 累积的调度开销越大。

此外, 基准版本的 stepM=4、stepN=5, 意味着每次迭代在 M 和 N 方向分别步进 4 和 5 个 base 块。较大的 step 值会影响数据在内存中的访问模式。基准版本也未设置遍历顺序, 使用框架的默认遍历策略。

2. 优化方案

Tiling 切分策略的优化包含三个方面。

(1) 增大 baseM 和 baseN。将 baseM 从 64 增大到 128 (或 256), baseN 从 64 增大到 128。增大分块尺寸意味着每次迭代处理更多数据, 迭代次数相应减少。以 baseM=128、baseN=128 为例, 迭代次数从 80 次减少到 $\frac{512}{128} \times \frac{640}{128} = 20$ 次, 减少了 75%。每次迭代的计算量增大后, 计算时间在总时间中的占比提高, 调度开销的比重下降。

需要注意的是, 增大分块尺寸要求 UB 有足够的空间容纳 $\text{baseM} \times \text{baseN}$ 个 float32 元素。对于 baseM=128、baseN=128, 需要 $128 \times 128 \times 4 = 65,536$ 字节。如果 PReLU 使用 4 条指令实现, 还需要同等大小的 tmpBuf 空间, 合计约 128KB。当分块进一步增大到 baseM=256 时, 4 条指令版本所需的 tmpBuf 可能超出 UB 容量。而使用 LeakyRelu 单指令后不需要 tmpBuf, 为增大分块尺寸提供了空间上的条件。

(2) 设置 FIRSTM 遍历顺序。将遍历模式设为 FIRSTM (M 方向优先), 同时将 CopyOut 中的偏移计算改为 M 优先的方式, 使遍历顺序与写回顺序一致。M 优先遍历在处理行主序存储的矩阵时, 可以获得更好的内存访问局部性。

(3) 减小 step 值。将 stepM 和 stepN 从 4/5 减小为 1/1。step=1 意味着每次迭代按顺序处理相邻的数据块, 避免了大步长跳跃带来的访存不连续。

(三) 优化方案的组合

本文设计了 5 个递进的实验版本，如表 2 所示。每个版本在前一个版本的基础上叠加一种优化手段，以便单独观察每种优化的贡献。

表 2: 5 个实验版本的参数配置

版本	描述	PReLU 实现	baseM	baseN	stepM/N	遍历顺序
V0	基准版本	4 条指令	64	64	4/5	默认
V1	指令优化	LeakyRelu	64	64	4/5	默认
V2	Tiling 优化	4 条指令	128	128	1/1	FIRSTM
V3	组合优化	LeakyRelu	128	128	1/1	FIRSTM
V4	全部优化	LeakyRelu	256	128	1/1	FIRSTM

V1 仅在 V0 基础上替换了 PReLU 的实现方式，其余参数不变。V2 仅调整了 Tiling 参数，PReLU 仍使用 4 条指令。V3 将两种优化结合。V4 在 V3 的基础上进一步增大 baseM 到 256，由于 LeakyRelu 不需要 tmpBuf，UB 空间可以支撑更大的分块。

五、实验与分析

(一) 实验环境

实验在华为昇腾 910B 处理器上进行，具体环境配置见表 3。

表 3: 实验环境配置

项目	配置
处理器	华为昇腾 910B1
CANN 版本	社区版
编程框架	Ascend C
编译选项	C++17, -O2
AI Core 数量	2 (实验使用)
矩阵规模	M=1024, N=640, K=256
输入数据类型	float16
输出数据类型	float32
PReLU 参数	$\alpha = 0.5$

每组实验先进行 3 次预热运行，消除首次启动的冷启动效应，然后正式测量 10 次，取平均值作为最终结果。计时使用 aclrtEvent 事件机制，精确测量 kernel 在 NPU 上的执行时间。精度验证使用 numpy 在 CPU 上计算参考结果，与算子输出进行逐元素对比，相对误差容忍阈值为 10^{-4} 。

(二) 实验结果

5 个版本的实验结果见表 4。

表 4: 各版本实验结果

版本	平均耗时 (ms)	有效算力 (TFLOPS)	相比 V0 加速	精度验证
V0 (基准)	0.4154	0.8077	—	通过
V1 (指令优化)	0.3779	0.8879	9.03%	通过
V2 (Tiling 优化)	0.2787	1.2039	32.91%	通过
V3 (组合优化)	0.2511	1.3364	39.55%	通过
V4 (全部优化)	0.2311	1.4518	44.37%	通过

有效算力的计算公式为：

$$\text{TFLOPS} = \frac{2 \times M \times N \times K}{t \times 10^{12}} \quad (3)$$

其中 t 为 kernel 执行时间 (秒)， $2 \times M \times N \times K$ 为矩阵乘法的浮点运算量（每个输出元素需要 K 次乘法和 K 次加法）。

(三) 性能分析

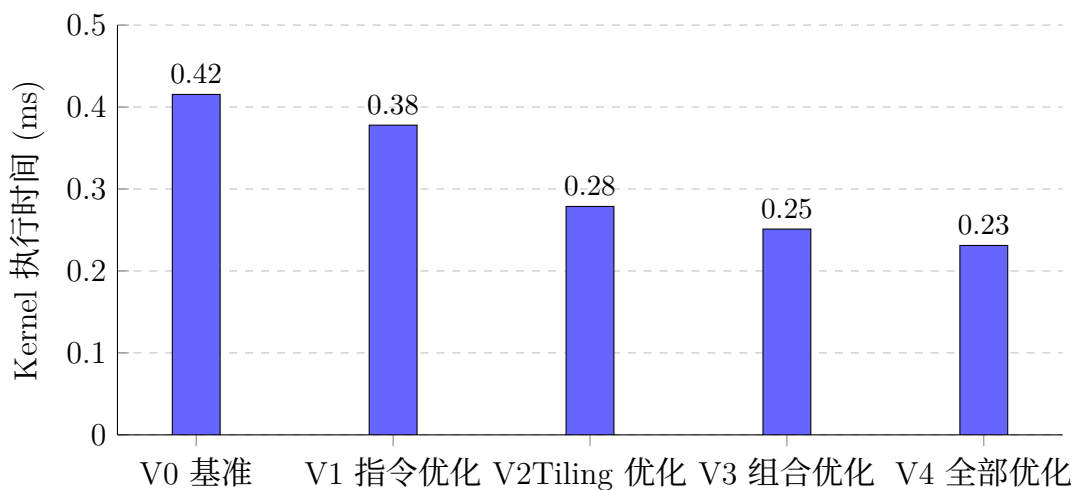


图 2: 各版本 Kernel 执行时间对比

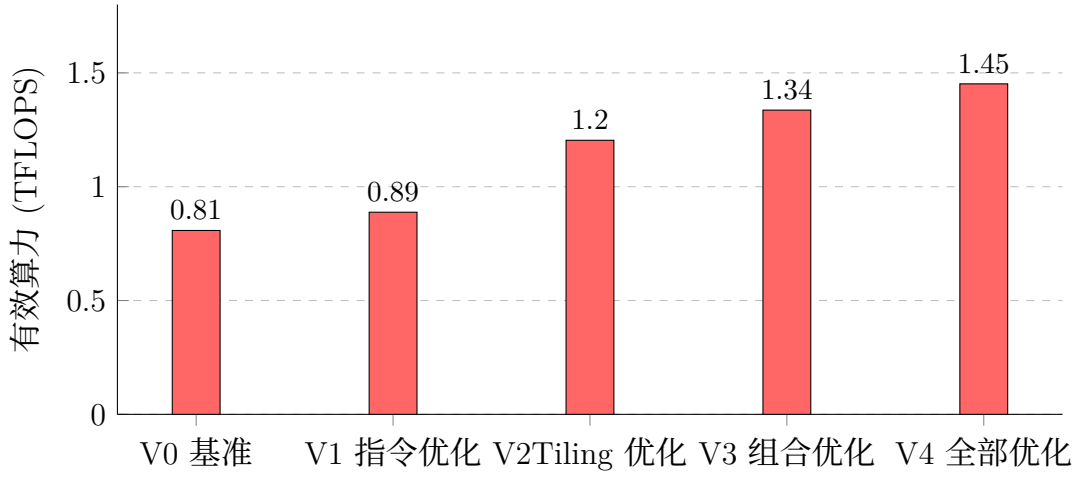


图 3: 各版本有效算力对比

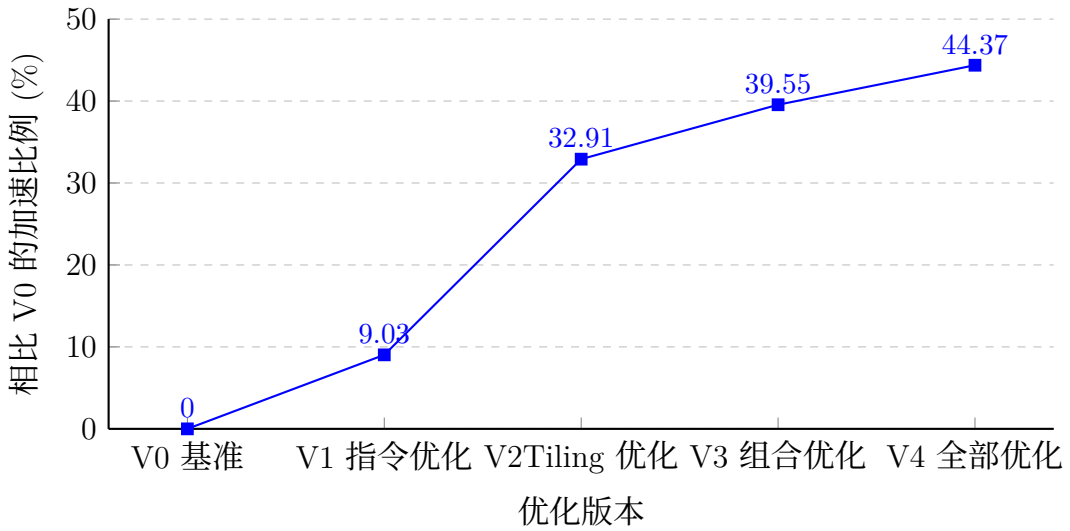


图 4: 各版本相比基准的加速比例

从实验结果可以得出以下分析。

(1) 向量指令优化的效果。对比 V0 和 V1，两者的 Tiling 参数完全相同，唯一的区别是 PReLU 的实现方式。V1 将 4 条向量指令替换为单条 LeakyRelu 指令后，执行时间从 0.4154ms 降低到 0.3779ms，减少了 9.03%。这一提升来自两方面：减少了 3 条向量指令的发射和流水线开销，以及消除了 tmpBuf 临时空间的分配和访问。

(2) Tiling 切分策略的效果。对比 V0 和 V2，两者的 PReLU 实现方式相同（均为 4 条指令），区别在于 Tiling 参数。V2 将 baseM/baseN 从 64/64 增大到 128/128，同时设置了 FIRSTM 遍历和 step=1。执行时间从 0.4154ms 降低到 0.2787ms，减少了 32.91%。这是最显著的一项优化。分析原因，基准版本的迭代次数为 $\frac{512}{64} \times \frac{640}{64} = 80$ 次，优化后减少为 $\frac{512}{128} \times \frac{640}{128} = 20$ 次。迭代次数减少了 75%，大幅降低了队列操作和 DMA 搬运指令的调度开销。FIRSTM 遍历顺序和 step=1 的设置也改善了内存访问的连续性。

(3) 组合优化的效果。V3 同时应用了指令优化和 Tiling 优化，执行时间为 0.2511ms，比单独应用 Tiling 优化的 V2 (0.2787ms) 又减少了 9.90%。两种优化手段的效果可以

叠加。

(4) 进一步增大分块尺寸的效果。V4 将 baseM 从 128 增大到 256，执行时间进一步降低到 0.2311ms。这一步之所以可行，是因为 LeakyRelu 单指令不需要 tmpBuf 空间，UB 中有足够的空间容纳 $256 \times 128 \times 4 = 131,072$ 字节的输出数据。如果仍然使用 4 条指令的 PReLU 实现，baseM=256 时需要额外的 131KB tmpBuf，可能超出 UB 容量限制。

(5) 各优化手段的贡献分析。图 5 展示了从 V0 到 V4 的总耗时降低中，各优化手段的贡献。总耗时降低为 $0.4154 - 0.2311 = 0.1843\text{ms}$ 。其中，指令优化贡献了 0.0375ms (占 20.4%)，Tiling 切分参数优化贡献了 0.0992ms (占 53.8%)，分块尺寸进一步增大贡献了 0.0476ms (占 25.8%)。Tiling 切分策略的优化贡献最大。这一结果说明，在 Ascend C 算子开发中，Tiling 参数的合理设定对性能的影响往往大于单条指令的优化。

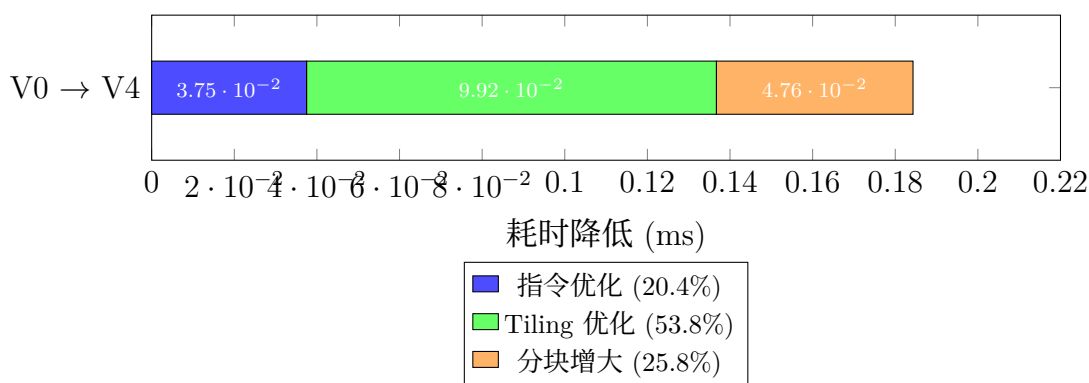


图 5: 各优化手段对耗时降低的贡献

(四) 优化前后的代码对比

表 5 总结了基准版本 (V0) 与最终版本 (V4) 在代码层面的主要区别。

表 5: V0 与 V4 的代码差异

对比项	V0 (基准)	V4 (最终)
PReLU 实现	Mins+Muls+Maxs+Add (4 条)	LeakyRelu (1 条)
临时缓冲区	需要 tmpBuf	不需要
baseM $\times$ baseN	$64 \times 64 = 4096$	$256 \times 128 = 32768$
迭代次数/核	80 次	10 次
stepM / stepN	4 / 5	1 / 1
遍历顺序	默认	FIRSTM
CopyOut 偏移	N 方向优先	M 方向优先

六、 讨论

(一) Tiling 参数选择的权衡

实验结果表明,Tiling 分块大小并不是越小越好。从 V0(baseM=64)到 V2(baseM=128),增大分块尺寸后性能提升了 32.91%。这是因为小分块导致迭代次数增多,每次迭代中 Tensor 分配/释放、队列入队/出队等操作的固定开销被放大。

但分块尺寸也不能无限增大,受到 UB 容量的限制。当 PReLU 使用 4 条指令实现时,需要额外的 tmpBuf 空间,这限制了 baseM 的上限。使用 LeakyRelu 单指令后消除了这一限制,使得 baseM 可以进一步增大到 256。这说明向量指令的优化和 Tiling 策略之间存在相互影响的关系:指令优化释放了片上资源,为 Tiling 策略提供了更大的调优空间。

(二) 遍历顺序和 step 参数的影响

本文的实验中,Tiling 优化同时包含了三个改动(增大 baseM/baseN、设置 FIRSTM、降低 step),没有对三者分别进行独立的对比实验。因此无法精确区分每个改动的单独贡献。从原理上分析,FIRSTM 遍历在行主序矩阵上可以获得更好的空间局部性,step=1 保证了相邻迭代处理连续的数据块,这两者都有利于降低内存访问延迟。

(三) 与理论峰值的差距

昇腾 910B 处理器的 float16 矩阵乘法理论峰值算力很高,本文实验中的最佳结果为 1.45 TFLOPS,与理论峰值之间仍有较大差距。这主要有以下原因:实验仅使用了 2 个 AI Core (处理器共有数十个);矩阵规模 ($1024 \times 640 \times 256$) 相对较小,未能充分利用硬件的并行能力;本文未采用更高级的优化技术如 DoubleBuffer 流水、L2 Cache 切分、IBShare 共享 B 矩阵等^[7]。这些方向可以作为后续工作的优化目标。

(四) 局限性

本文的实验存在以下局限:(1) 仅在单一矩阵规模 ($1024 \times 640 \times 256$) 上进行了实验,未验证不同规模下优化效果的变化。(2) 核数固定为 2,未测试多核扩展性。(3) PReLU 参数 α 固定为 0.5,使用了 LeakyRelu 接口替代,未讨论 α 为可学习参数时的实现方案。(4) 未与 GPU 上的等价实现进行跨平台对比。

七、 结语

本文在华为昇腾 910B 处理器上,基于 Ascend C 编程框架实现了 MatMul+PReLU 融合算子,并从向量指令和 Tiling 切分策略两个方面进行了系统的性能优化。实验结果表明:(1) 将 4 条 PReLU 向量指令替换为单条 LeakyRelu 内置指令,可以减少 9.03% 的 kernel 执行时间,同时释放了片上临时缓冲区空间。(2) 通过增大 Tiling 分块尺寸

(baseM 从 64 增大到 128)、设置 FIRSTM 遍历顺序和降低 step 值,可以减少 32.91% 的执行时间。Tiling 切分策略是影响性能的主要因素。(3) 将两种优化结合后,最终版本(V4, baseM=256/baseN=128+LeakyRelu)相比基准版本加速了 1.80 倍(从 0.4154ms 降低到 0.2311ms),有效算力从 0.81 TFLOPS 提升到 1.45 TFLOPS,提升了 79.75%。(4) 向量指令优化和 Tiling 策略之间存在协同关系:指令优化释放的片上资源使得更大的 Tiling 分块成为可能,两者结合可以获得比单独应用更好的效果。

本文的实验数据和优化经验可为昇腾平台上的自定义算子开发提供参考。后续工作可以从多核扩展、DoubleBuffer 流水优化、L2 Cache 切分等方向进一步提升算子性能。

参考文献

- [1] He K, Zhang X, Ren S, et al. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification[C]. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2015: 1026–1034.
- [2] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional Architecture for Fast Feature Embedding[C]. Proceedings of the 22nd ACM International Conference on Multimedia, 2014: 675–678.
- [3] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[C]. Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018: 578–594.
- [4] Vasilache N, Zinenko O, Theodoridis T, et al. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions[J]. arXiv preprint arXiv:1802.04730, 2018.
- [5] 华为技术有限公司. Ascend C 算子开发指南 [EB/OL]. CANN 社区版文档, 2026. <https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/>.
- [6] 华为技术有限公司. 算子实践参考——融合算子编程 [EB/OL]. CANN 社区版 Ascend C 算子开发指南, 2026.
- [7] 华为技术有限公司. SIMD 算子性能优化 [EB/OL]. CANN 社区版 Ascend C 算子开发指南, 2026.